



Семинар МЕТА, ИПС им. А.К. Айламазяна РАН

Конструирование абстрактного интерпретатора JavaScript

От решёток до абстрактной машины CESKT

Илья Афанасьев
affeeal@ya.ru

Что мы хотим узнать о программе?

Один запуск

Тестирование проверяет одну конкретную трассу:

$input \mapsto output.$

Даже большой набор тестов оставляет непроверенные входы и пути.

Все исполнения

Статический анализ проверяет сразу множество трасс:

- какие значения возможны;
- достижима ли ошибка;
- сохраняется ли инвариант;
- возможно ли незавершение.

Абстрактная интерпретация заменяет потенциально бесконечное множество исполнений конечным корректным описанием.



Мотивирующий пример

Программа

```
let x = 0;
while (x < 100) {
  x += unknownBoolean()
    ? -2
    : 3;
}

let o =
  unknownBoolean()
    ? { limit: 100 }
    : null;
o.limit = x;
```

Результат анализа

- цикл может выполняться бесконечно;
- при выходе из цикла

$$x \in [100, 102];$$

- при обращении к o потенциально возможен `TypeError`.



Частично упорядоченное множество

Определение

Пара $\langle L, \sqsubseteq \rangle$ является **частично упорядоченным множеством** (ЧУМ), если для любых $x, y, z \in L$ выполняются

$$x \sqsubseteq x \quad (\text{рефлексивность}),$$

$$x \sqsubseteq y \wedge y \sqsubseteq x \Rightarrow x = y \quad (\text{антисимметричность}),$$

$$x \sqsubseteq y \wedge y \sqsubseteq z \Rightarrow x \sqsubseteq z \quad (\text{транзитивность}).$$

Пример

$\langle \mathbb{N}, \leq \rangle$ является *полностью* упорядоченным множеством: любые два натуральных числа сравнимы.

Определим булеан (англ. powerset)

$$\wp(X) := \{Y \mid Y \subseteq X\}.$$

ЧУМ $\langle \wp(\mathbb{N}), \subseteq \rangle$ упорядочено лишь частично: $\{1\}$ и $\{2\}$ несравнимы.



Верхние и нижние грани

Определение

Пусть $X \subseteq L$. Элемент $u \in L$ называется **верхней гранью** X , если

$$\forall x \in X : x \sqsubseteq u.$$

Наименьшая верхняя грань, если существует, называется **супремумом** X и обозначается $\sqcup X$.

Элемент $l \in L$ называется **нижней гранью** X , если

$$\forall x \in X : l \sqsubseteq x.$$

Наибольшая нижняя грань называется **инфимумом** X и обозначается $\sqcap X$.

$\perp$ — **наименьший** элемент (англ. bottom), если $\forall x \in L : \perp \sqsubseteq x$. Двойственно, $\top$ — **наибольший** элемент (англ. top), если $\forall x \in L : x \sqsubseteq \top$.



Операции супремума и инфимума

Определение

Введём частичную бинарную функцию

$$\sqcup : D_{\sqcup} \rightarrow L, \quad D_{\sqcup} := \{(x, y) \in L^2 \mid \bigsqcup\{x, y\} \text{ существует}\},$$

$$x \sqcup y := \bigsqcup\{x, y\}.$$

Двойственно определяются

$$\sqcap : D_{\sqcap} \rightarrow L, \quad x \sqcap y := \bigsqcap\{x, y\}.$$

Операции называют **join** и **meet**.



Полурешётки и решётки

Определение

ЧУМ является **верхней полурешёткой**, если $x \sqcup y$ существует для любых $x, y \in L$.

Определение

ЧУМ является **нижней полурешёткой**, если $x \sqcap y$ существует для любых $x, y \in L$.

Определение

Решётка является одновременно верхней и нижней полурешёткой. Она **ограничена**, если содержит $\perp$ и $\top$.



Пример: решётка булеана

Пример

Для заданного универсума U

$$\mathcal{L}_{\wp(U)} := \langle \wp(U), \subseteq, \cup, \cap, \emptyset, U \rangle :$$

$$X \sqcup Y = X \cup Y, \quad X \sqcap Y = X \cap Y, \quad \perp = \emptyset, \quad \top = U.$$

В этой решётке порядок совпадает с включением множеств, супремум — с объединением, а инфимум — с пересечением.



Алгебраическое определение решётки

Определение

Решётка — алгебраическая структура $\langle L, \sqcup, \sqcap \rangle$, в которой обе операции коммутативны, ассоциативны и удовлетворяют законам поглощения:

$$x \sqcup (x \sqcap y) = x, \quad x \sqcap (x \sqcup y) = x.$$

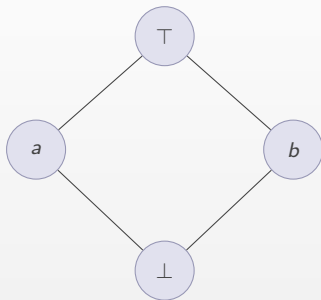
Из этих свойств следуют идемпотентность и восстановление порядка:

$$x \sqcup x = x, \quad x \sqcap x = x,$$

$$x \sqsubseteq y \iff x \sqcup y = y \iff x \sqcap y = x.$$



Диаграмма Хассе

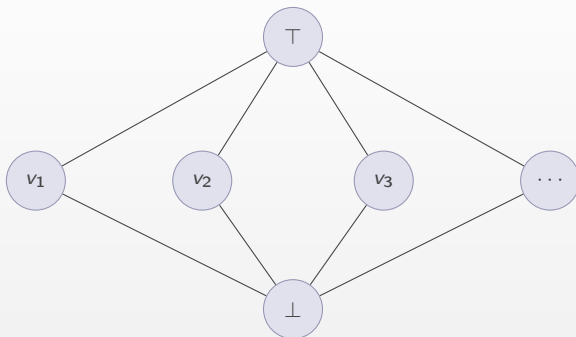


Как читать

- выше означает «не меньше»;
- транзитивные рёбра не рисуются;
- a и b несравнимы;
- $a \sqcup b = T$;
- $a \sqcap b = \perp$.



Плоская решётка

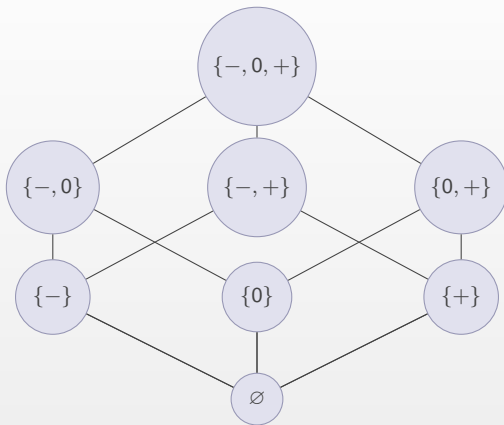


Промежуточные элементы попарно несравнимы; для $i \neq j$

$$v_i \sqcup v_j = \top, \quad v_i \sqcap v_j = \perp.$$



Решётка знаков



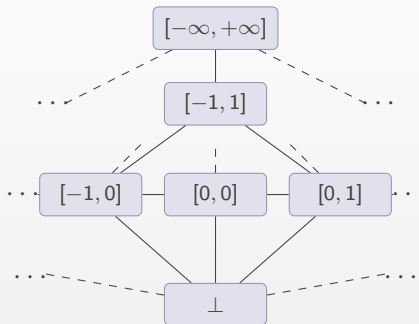
Бесконечная решётка интервалов

Определение

$\text{Interval} := \{\perp\} \cup$
 $\{[l, u] \mid l, u \in \overline{\mathbb{Z}}, l \leq u\},$
 $\overline{\mathbb{Z}} := \mathbb{Z} \cup \{-\infty, +\infty\},$
 $[l_1, u_1] \subseteq [l_2, u_2] \iff$
 $l_2 \leq l_1 \wedge u_1 \leq u_2.$

Пример

$[0, 2] \sqcup [5, 7] = [0, 7],$
 $[0, 4] \sqcap [2, 5] = [2, 4].$



Многоточия обозначают остальные интервалы.



Прямое произведение решёток

Пусть I — конечное непустое множество индексов, а $\langle L_i, \sqsubseteq_i, \sqcup_i \rangle, i \in I$, — верхние полурешётки. Их прямое произведение имеет носитель

$$L_{\times} := \prod_{i \in I} L_i$$

и покомпонентные порядок и супремум:

$$x \sqsubseteq_{\times} y \iff \forall i \in I : x_i \sqsubseteq_i y_i,$$

$$x \sqcup_{\times} y := (x_i \sqcup_i y_i)_{i \in I}.$$

Если компоненты ограничены, то

$$\perp_{\times} = (\perp_i)_{i \in I}, \quad \top_{\times} = (\top_i)_{i \in I}.$$



Поточечное расширение

Пусть K — произвольное множество ключей, а L — верхняя полурешётка. Тогда отображения $K \rightarrow L$ образуют верхнюю полурешётку:

$$f \sqsubseteq g \iff \forall k \in K : f(k) \sqsubseteq g(k), \quad (f \sqcup g)(k) := f(k) \sqcup g(k).$$



Определение

Цепь $C \subseteq L$ — подмножество, любые два элемента которого сравнимы.

Индексированная цепь — последовательность $\{x_i\}_{i \in \mathbb{N}}$, множество элементов которой образует цепь. Она **не убывает**, если

$$i < j \Rightarrow x_i \sqsubseteq x_j,$$

и **строго возрастает**, если дополнительно $x_i \neq x_j$.

Определение

ЧУМ нётерово, или удовлетворяет условию обрыва неубывающих цепей (англ. ascending chain condition, ACC), если всякая неубывающая индексированная цепь стабилизируется.



Неподвижные и префиксные точки

Определение

Для функции $F : L \rightarrow L$:

$F(x) = x$ — неподвижная точка,

$F(x) \sqsubseteq x$ — префиксная точка,

$x \sqsubseteq F(x)$ — постфиксная точка.

$\text{lfp } F$ обозначает наименьшую среди всех неподвижных точек F , если она существует.



ПЧП и теорема Клини

Определение

Полный частичный порядок (ПЧП; англ. complete partial order, CPO) содержит $\perp$, а всякая неубывающая индексированная цепь $\{x_i\}$ имеет супремум $\bigsqcup_i x_i$.

Определение

Функция $F : L \rightarrow L$ сохраняет супремумы цепей, если

$$F\left(\bigsqcup_i x_i\right) = \bigsqcup_i F(x_i)$$

для всякой неубывающей индексированной цепи $\{x_i\}$.

Теорема (Клини)

Если $F : L \rightarrow L$ сохраняет супремумы цепей на ПЧП L , то

$$\text{lfp } F = \bigsqcup_{n \in \mathbb{N}} F^n(\perp).$$



Абстракция как смена представления

Конкретный домен

$$C := \langle \wp(\mathbb{Z}), \subseteq \rangle.$$

Он точно представляет любое множество целых чисел, но имеет бесконечно много элементов.

Абстрактный домен

$$A := \langle \text{Interval}, \sqsubseteq \rangle.$$

Интервал имеет конечное представление, но может содержать лишние числа.



Надприближение (англ. overapproximation) может добавлять конкретные элементы, но не должно терять исходные.



Соединение Галуа

Пусть $\langle C, \sqsubseteq_C \rangle$ и $\langle A, \sqsubseteq_A \rangle$ — ЧУМ конкретных и абстрактных элементов.

Определение

$\alpha : C \rightarrow A$ — неубывающая функция абстракции,

$\gamma : A \rightarrow C$ — неубывающая функция конкретизации.

Они образуют **соединение Галуа**, если

$$\alpha(c) \sqsubseteq_A a \iff c \sqsubseteq_C \gamma(a).$$

Пример

$$\alpha(\{1, 3, 7\}) = [1, 7], \quad \gamma([1, 7]) = \{1, \dots, 7\}.$$



Корректная и наилучшая абстракции

Определение

Абстрактный элемент a **корректно** (англ. sound) представляет конкретный элемент c , если

$$c \sqsubseteq_C \gamma(a).$$

Сравнение точности

$$a_1 \sqsubseteq_A a_2 \Rightarrow \gamma(a_1) \sqsubseteq_C \gamma(a_2).$$

a_1 не менее точен, чем a_2 .

Наилучшая абстракция

$$\alpha(c) = \min_{\sqsubseteq_A} \{a \mid c \sqsubseteq_C \gamma(a)\}.$$

$\alpha(c)$ — наиболее точный корректный элемент.



Пример: нотация $\mathcal{O}$

Пусть конкретный элемент — функция $f : \mathbb{N} \rightarrow \mathbb{R}$, а абстрактный элемент $\mathcal{O}(g)$ задаёт класс её роста:

$$\gamma(\mathcal{O}(g)) := \{f \mid \exists c > 0, \exists n_0, \forall n \geq n_0 : |f(n)| \leq c|g(n)|\}.$$

$$3n + 2 \in \gamma(\mathcal{O}(n)), \quad \mathcal{O}(1) \sqsubseteq \mathcal{O}(n) \sqsubseteq \mathcal{O}(n^2).$$

Абстракция забывает коэффициенты и поведение на конечном числе аргументов, сохраняя асимптотический рост.



Корректность абстрактной операции

Определение

Пусть заданы $f : C^n \rightarrow C$ и $f^\# : A^n \rightarrow A$. Оператор $f^\#$ корректно абстрагирует f , если

$$\{f(c_1, \dots, c_n) \mid c_i \in \gamma(a_i)\} \subseteq \gamma(f^\#(a_1, \dots, a_n)).$$

Пример

Пусть $C = \wp(\mathbb{Z})$, $A = \text{Interval}$, $\gamma([l, u]) = \{z \in \mathbb{Z} \mid l \leq z \leq u\}$, а конкретная операция — сложение множеств чисел. Тогда

$$[l_1, u_1] +^\# [l_2, u_2] := [l_1 + l_2, u_1 + u_2],$$

$$[1, 3] +^\# [10, 20] = [11, 23].$$



Прямое произведение абстракций

Пусть I — конечное непустое множество индексов, $\langle C, \sqsubseteq_C \rangle$ — нижняя полурешётка и для каждого $i \in I$ задано соединение Галуа

$$\langle C, \sqsubseteq_C \rangle \stackrel{\alpha_i}{\underset{\gamma_i}{\cong}} \langle A_i, \sqsubseteq_i \rangle.$$

Теорема

Прямое произведение $A_\times := \prod_{i \in I} A_i$ образует соединение Галуа с тем же конкретным доменом C :

$$\alpha_\times(c) := (\alpha_i(c))_{i \in I},$$

$$\gamma_\times(a) := \prod_{i \in I} \gamma_i(a_i).$$

Компоненты совместно и независимо описывают один конкретный элемент.



Дизъюнктивное объединение абстракций

Пусть I — конечное и непустое множество индексов, множества V_i попарно не пересекаются, $V_{\sqcup} := \bigcup_{i \in I} V_i$, и для каждого i задано

$$\langle \wp(V_i), \subseteq \rangle \xrightarrow[\gamma_i]{\alpha_i} \langle A_i, \sqsubseteq_i \rangle.$$

Теорема

Домен $A_{\sqcup} := \prod_{i \in I} A_i$ образует соединение Галуа с $\wp(V_{\sqcup})$:

$$\gamma_{\sqcup}(a) := \bigcup_{i \in I} \gamma_i(a_i),$$

$$\alpha_{\sqcup}(S) := (\alpha_i(S \cap V_i))_{i \in I}.$$

Каждая компонента описывает отдельный вид конкретных значений.



Редуцированное произведение

Определение

Для произведения $A_{\times}$ оператор **редукции** $\rho : A_{\times} \rightarrow A_{\times}$ не убывает, идемпотентен, редуктивен и сохраняет конкретизацию:

$$\begin{aligned}a \sqsubseteq b &\Rightarrow \rho(a) \sqsubseteq \rho(b), & \rho(\rho(a)) &= \rho(a), \\ \rho(a) &\sqsubseteq a, & \gamma_{\times}(\rho(a)) &= \gamma_{\times}(a).\end{aligned}$$

Пример

Пусть $A_{\times} = \text{Interval} \times \text{Sign}$ совместно описывает одно целое число. Элемент

$$\langle [0, 10], \text{Negative} \rangle$$

не описывает ни одного числа. Сохраняющая конкретизацию редукция приводит его к $\perp_{\times}$.



Определение

Семантическая функция

$$S : \text{Program} \rightarrow D$$

сопоставляет программе элемент семантического домена D , описывающий её поведение.

Далее рассматривается *семантика достижимости* на основе операционной семантики малых шагов.

Денотационная

Математические функции и значения.

Аксиоматическая

Логические утверждения о выполнении.

Операционная

Переходы между состояниями.



Определение

Пусть Σ — множество состояний вычислительной машины. Для программы P задаются

$I_P \subseteq \Sigma$ — начальные состояния,

$\longrightarrow \subseteq \Sigma \times \Sigma$ — отношение одного шага.

Запись $\sigma \longrightarrow \sigma'$ означает, что машина может за один шаг перейти из σ в σ' . Отношение может быть недетерминированным: одному состоянию соответствует несколько преемников.



Трассы и достижимость

Определение

Трасса исполнения — конечная или бесконечная последовательность

$$\sigma_0 \longrightarrow \sigma_1 \longrightarrow \sigma_2 \longrightarrow \cdots, \quad \sigma_0 \in I_P.$$

Достижимое состояние

σ достижимо, если

$$\exists \sigma_0 \in I_P : \sigma_0 \longrightarrow^* \sigma.$$

Незавершение

Бесконечная трасса представляет исполнение, не достигающее завершающего состояния.



Конкретный оператор достижимости

Распространим отношение одного шага на множества состояний:

$$\text{post} : \wp(\Sigma) \rightarrow \wp(\Sigma),$$

$$\text{post}(X) := \{\sigma' \mid \exists \sigma \in X : \sigma \longrightarrow \sigma'\}.$$

Определение

Оператор достижимости программы P :

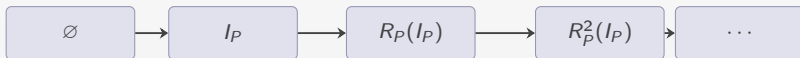
$$R_P : \wp(\Sigma) \rightarrow \wp(\Sigma), \quad R_P(X) := I_P \cup \text{post}(X).$$

post и R_P сохраняют объединения неубывающих цепей.



Конкретная семантика достижимости

$$\mathcal{S}_R[P] := \text{lfp } R_P = \bigcup_{n \in \mathbb{N}} R_P^n(\emptyset).$$



Каждое приближение добавляет состояния, достижимые ещё за один шаг. В общем случае точное множество достижимых состояний невычислимо: разрешение достижимости завершения решало бы проблему останова.



Абстрактный домен состояний

Выберем эффективно представимую ограниченную верхнюю полурешётку

$$\widehat{\mathcal{R}} := \langle \widehat{\Sigma}, \sqsubseteq, \sqcup, \perp, \top \rangle$$

и неубывающую конкретизацию

$$\gamma : \widehat{\Sigma} \rightarrow \wp(\Sigma).$$

Корректность супремума

Из неубывания γ и свойств супремума следует

$$\gamma(\hat{\sigma}_1) \cup \gamma(\hat{\sigma}_2) \subseteq \gamma(\hat{\sigma}_1 \sqcup \hat{\sigma}_2).$$

Для построения корректного анализа функция α и полная решётка не обязательны.



Корректный абстрактный переход

Пусть

$$\widehat{\longrightarrow} \subseteq \widehat{\Sigma} \times \widehat{\Sigma}$$

— вычислимое абстрактное отношение перехода с конечным числом приемников каждого состояния.

Условие корректности

$$\begin{aligned} \sigma \in \gamma(\hat{\sigma}) \wedge \sigma \longrightarrow \sigma' \\ \Rightarrow \exists \hat{\sigma}' : \hat{\sigma} \widehat{\longrightarrow} \hat{\sigma}' \wedge \sigma' \in \gamma(\hat{\sigma}'). \end{aligned}$$

Определим неубывающий оператор

$$\widehat{\text{post}}(\hat{\sigma}) := \bigsqcup \{ \hat{\sigma}' \mid \hat{\sigma} \widehat{\longrightarrow} \hat{\sigma}' \}.$$

Тогда $\text{post}(\gamma(\hat{\sigma})) \subseteq \gamma(\widehat{\text{post}}(\hat{\sigma}))$.



Абстрактная семантика достижимости

Пусть $\hat{l}_P \in \hat{\Sigma}$ корректно представляет начальные состояния: $l_P \subseteq \gamma(\hat{l}_P)$.
Определим

$$\hat{R}_P(\hat{\sigma}) := \hat{l}_P \sqcup \widehat{\text{post}}(\hat{\sigma}).$$

Тогда

$$R_P(\gamma(\hat{\sigma})) \subseteq \gamma(\hat{R}_P(\hat{\sigma})).$$

Теорема

Если a^* является префиксной точкой:

$$\hat{R}_P(a^*) \sqsubseteq a^*,$$

то она корректно абстрагирует семантику достижимости:

$$S_R[P] \subseteq \gamma(a^*).$$



$$\text{State}_{\text{СЕК}} := \text{Control} \times \text{Env} \times \text{Kont}.$$

Управляющий элемент

Окружение

Продолжение

Control — вычисляемая синтаксическая конструкция либо уже полученное значение.

Environment — структура, связывающая имена со значениями языка.

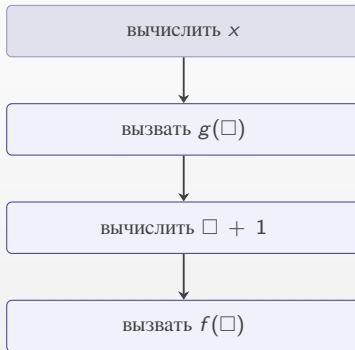
[C/K]ontinuation — явное представление оставшейся части вычисления.

Компоненты состояния изменяются локальными переходами семантики малых шагов.



Что такое продолжение?

$$f(g(x) + 1)$$



Продолжение — явная цепочка кадров, представляющая остаток вычисления, который обычный интерпретатор хранит в стеке.



$$\begin{aligned}\langle e_1 + e_2, \rho, \kappa \rangle &\longrightarrow \langle e_1, \rho, \text{AddL}(e_2, \rho, \kappa) \rangle, \\ \langle v_1, \rho, \text{AddL}(e_2, \rho', \kappa) \rangle &\longrightarrow \langle e_2, \rho', \text{AddR}(v_1, \kappa) \rangle, \\ \langle v_2, \rho, \text{AddR}(v_1, \kappa) \rangle &\longrightarrow \langle v_1 + v_2, \rho, \kappa \rangle.\end{aligned}$$

Этапы перехода

1. Вычислить левый операнд, сохранив правый в AddL.
2. Сохранить левое значение в AddR и вычислить правое.
3. Сложить значения и передать результат продолжению κ .



Условный оператор

$$\begin{aligned} &\langle \text{if } (e) \text{ s1 else s2}, \rho, \kappa \rangle \\ &\quad \longrightarrow \langle e, \rho, \text{Branch}(s1, s2, \rho, \kappa) \rangle, \\ &\langle v, \rho', \text{Branch}(s1, s2, \rho, \kappa) \rangle \\ &\quad \longrightarrow \begin{cases} \langle s1, \rho, \kappa \rangle, & \text{ToBoolean}(v) = \text{true}, \\ \langle s2, \rho, \kappa \rangle, & \text{ToBoolean}(v) = \text{false}. \end{cases} \end{aligned}$$

Сначала вычисляется условие e . Затем конкретная операция ToBoolean однозначно выбирает одну ветвь, а κ сохраняет вычисление после условного оператора.



Вызов функции JavaScript

$$\langle f(e_1, \dots, e_n), \rho, \kappa \rangle \\ \longrightarrow \langle f, \rho, \text{CallFun}(e_1, \dots, e_n, \rho, \kappa) \rangle.$$

Подготовка вызова

Кадры сохраняют вычисленную функцию, уже вычисленные аргументы, оставшиеся выражения и адрес возврата.

Вход в функцию

Создаётся окружение вызова. Внешнее окружение берётся из замыкания, параметры связываются с аргументами, а для метода устанавливается `this`.

`return` передаёт значение продолжению вызывающей стороны.



Почему СЕК трудно абстрагировать напрямую?

В состоянии JavaScript возникают взаимно рекурсивные структуры.

Объекты

Объект содержит свойства и ссылку `[[Prototype]]`; значения свойств снова могут быть объектами или функциями.

Окружения

Запись окружения хранит связывания и ссылку `[[OuterEnv]]` на внешнее окружение.

Продолжения

Кадр содержит остаток вычисления, окружение и следующий кадр; функции замыкают окружения.

Рекурсивные значения неудобны для непосредственного определения операций абстрактного домена. Их рекурсию можно представить графом адресов.



Машина CESH: рекурсия через адреса

$$\text{State}_{\text{CESH}} := \text{Control} \times \text{Addr}_{\text{Env}} \times \text{Store} \times \text{Addr}_{\text{Kont}},$$
$$\text{Store} := \text{Store}_{\text{Obj}} \times \text{Store}_{\text{Env}} \times \text{Store}_{\text{Kont}}.$$

Объекты

Окружения

Продолжения

$\text{Addr}_{\text{Obj}} \rightarrow \text{Object}.$

$\text{Addr}_{\text{Env}} \rightarrow \text{Environment}.$

$\text{Addr}_{\text{Kont}} \rightarrow \text{Continuation}.$

В хранилище размещены все три рекурсивные категории, а состояние содержит только их адреса.



$$\text{State}_{\text{CESKT}} := \text{Control} \times \text{Addr}_{\text{Env}} \times \text{Store} \times \text{Addr}_{\text{Kont}} \times \text{Time}.$$

Параметры машины

$$\text{tick} : \text{State} \times \text{TickContext} \rightarrow \text{Time},$$
$$\text{alloc} : \text{State} \times \text{AllocationContext} \rightarrow \text{Addr}.$$

- tick выбирает следующую временную метку;
- alloc выбирает адрес размещаемой сущности.



Зачем ограничивать адреса и время?

Конкретная машина

- свежий адрес для каждого размещения;
- неограниченная история исполнения;
- потенциально бесконечный граф состояний.

Абстрактная машина

- конечное $\widehat{\text{Addr}}$;
- конечное $\widehat{\text{Time}}$;
- совпавшие размещения объединяются;
- при конечных доменах граф состояний конечен.

Конечность делает достижимость разрешимой. Для доменов с бесконечными цепями, например интервалов, завершение дополнительно обеспечивается widening.



Временная метка и чувствительность

Места вызова

$$t := [\ell_1, \dots, \ell_k].$$

$k = 0$ соответствует 0-CFA; большее k различает последние места вызова.

Входы в циклы

$$t := [h_1, \dots, h_m].$$

История входов различает первые развёртки цикла и уточняет инварианты.

Получатели методов

$$t := [o_1, \dots, o_r].$$

История получателей разделяет вызовы одного метода на разных объектах.

Компоненты времени объединяются прямым произведением.
Повышение чувствительности увеличивает точность и число состояний.



Адреса и слабые обновления

После выбора конечного времени адрес определяется контекстом размещения:

$$\hat{a} := \langle \text{AllocationContext}, \hat{t} \rangle.$$

Если $\hat{a}$ представляет несколько конкретных размещений, требуется слабое обновление:

$$\hat{\sigma}' := \hat{\sigma}[\hat{a} \mapsto \hat{\sigma}(\hat{a}) \sqcup \hat{v}].$$

Чем меньше множество адресов, тем чаще разные размещения объединяются и тем больше ложных связей возникает между значениями.



Структурная абстракция машины

Конкретное хранилище

Адрес однозначно определяет конкретный объект, окружение или продолжение; ячейка содержит одно значение.

Абстрактное хранилище

Один адрес представляет несколько конкретных размещений; ячейка содержит их объединённое абстрактное описание.

$$\begin{aligned}\widehat{\text{Store}}_{\text{Obj}} &: \widehat{\text{Addr}}_{\text{Obj}} \rightarrow \widehat{\text{Object}}, \\ \widehat{\text{Store}}_{\text{Env}} &: \widehat{\text{Addr}}_{\text{Env}} \rightarrow \widehat{\text{Environment}}, \\ \widehat{\text{Store}}_{\text{Kont}} &: \widehat{\text{Addr}}_{\text{Kont}} \rightarrow \widehat{\text{Continuation}}.\end{aligned}$$



Вынесение глобального хранилища

В машине CESKT хранилище является компонентой каждого состояния. При анализе его копии для разных достижимых состояний можно объединить:

$$\text{System} := \langle \text{Reachable}, \widehat{\text{Store}} \rangle,$$

где Reachable накапливает состояния без отдельной копии хранилища.

Преимущество

Состояния больше не дублируются только из-за различий в накопленном хранилище.

Цена

Обновления всех путей сливаются в общих ячейках; слабые обновления требуются чаще.



Агрегированное значение JavaScript

Конкретные категории значений попарно не пересекаются:

$$\text{Value}_{\text{JS}} := \text{Undefined} \uplus \text{Null} \uplus \text{Boolean} \uplus \text{Number} \uplus \text{String} \uplus \text{ObjectRef}.$$

Их абстракции образуют дизъюнктивное объединение:

$$\widehat{\text{Value}}_{\text{JS}} := A_{\text{Undef}} \times A_{\text{Null}} \times A_{\text{Bool}} \times A_{\text{Num}} \times A_{\text{Str}} \times A_{\text{ObjRef}}.$$

Пример

$$\langle \perp, \text{null}, \perp, [1, 3], \{\text{x}\}, \perp \rangle$$

представляет значение, которое может быть `null`, числом из `[1, 3]` или строкой `x`.



Определение

Направленные операции Refine и Difference удовлетворяют условиям

$$\gamma(a) \cap \gamma(b) \subseteq \gamma(\text{Refine}(a, b)) \subseteq \gamma(a),$$

$$\gamma(a) \setminus \gamma(b) \subseteq \gamma(\text{Difference}(a, b)) \subseteq \gamma(a).$$

$$\text{IsSingleton}(b) \iff |\gamma(b)| = 1.$$

Первый аргумент уточняется вторым. Операции корректны и редуктивны, но Refine не обязана вычислять точный инфимум, а Difference — точную разность.



Уточнение ветвей исполнения

JavaScript

```
if (x !== 0) {  
    y = 10 / x;  
} else {  
    y = 0;  
}
```

Пример

Пусть перед условием уточнение связывания b_x равно

$$C(b_x) = [0, 20].$$

Тогда анализ строит два преемника:

$$C_{\text{true}}(b_x) = \text{Difference}([0, 20], [0, 0]),$$

$$C_{\text{true}}(b_x) = [1, 20],$$

$$C_{\text{false}}(b_x) = \text{Refine}([0, 20], [0, 0]) = [0, 0].$$

Операция ToBoolean аналогично фильтрует компоненты агрегированного значения отдельно для истинной и ложной ветвей.



Ключ и данные состояния

$$\text{StateKey} := \text{ProgramPoint} \times \text{Time} \times \text{Addr}_{\text{Env}} \times \text{Addr}_{\text{Kont}},$$
$$\text{StateData} := \text{Data} \times \text{Refinements}.$$

Путевые уточнения

$$\text{Refinements} : (\text{Addr}_{\text{Env}} \times \text{Name}) \rightarrow \widehat{\text{Value}}_{\text{JS}}.$$

Они хранят локальные факты, полученные из условий, не перезаписывая глобальное хранилище.

$$R : \text{StateKey} \rightarrow \text{StateData}, \quad \text{System} = \langle R, \widehat{\text{Store}} \rangle.$$


Итерация абстрактного оператора

Для программы P определён неубывающий оператор

$$\hat{R}_P : \text{System} \rightarrow \text{System}.$$

Его накопительная итерация имеет вид

$$S_0 := \perp, \quad S_{n+1} := S_n \sqcup \hat{R}_P(S_n).$$

В нётеровом домене последовательность стабилизируется.

Интервальный домен допускает бесконечную цепь

$$[0, 0] \sqsubseteq [0, 1] \sqsubseteq [0, 2] \sqsubseteq \dots$$



Widening гарантирует стабилизацию

Widening ∇ экстраполирует возрастающую последовательность:

$$[0, 0] \nabla [0, 1] = [0, +\infty].$$

Отложенное расширение

Первые d изменений обрабатываются супремумом. Widening применяется, только если значение продолжает возрастать.

Расширение с порогами

Границы переходят сначала к выбранным константам:

$$0, 10, 100, +\infty,$$

а не сразу к бесконечности.

Widening гарантирует завершение возрастающей фазы ценой возможной потери точности.



Narrowing возвращает часть точности

$$B_0 := S^*, \quad B_{n+1} := B_n \triangle \hat{R}_P(B_n).$$

Требования

- результат не больше текущего приближения: $B_{n+1} \sqsubseteq B_n$;
- реальные достижимые состояния не исключаются;
- число полных проходов ограничивается политикой анализа.

Пример

Пусть widening дал $x \in [0, +\infty]$, а повторное вычисление переходов даёт кандидата $x \in [0, 100]$. Интервальное narrowing может уточнить результат до $[0, 100]$.



Результат одного абстрактного перехода

$$\text{transfer}_P : \text{StateKey} \times \text{StateData} \times \text{StoreReader} \\ \rightarrow \text{TransferResult},$$
$$\text{TransferResult} := \text{StateUpdates} \times \text{StoreDelta} \times \text{Diagnostics}.$$

Обновления состояний

Конечный набор пар
 (k', p') , где

$$k' \in \text{StateKey},$$
$$p' \in \text{StateData}.$$

Обновления хранилища

Отложенные записи.
 transfer не изменяет
глобальное хранилище
напрямую.

Чтения

Логирование ячеек
хранилища, прочи-
танных при работе
 transfer_P .



Алгоритм с рабочим списком

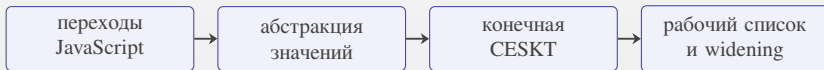
Псевдокод возрастающей фазы

```
W := {k0}, R(k0) := p0
считать R(k) = ⊥, если k ∉ dom R
пока W ≠ ∅:
    k := pop(W), p := R(k)
    reader := StoreReader( $\widehat{\text{Store}}$ )
    (U, Δ, G) := transferp(k, p, reader)
    Dependencies.replace(k, reader.Reads)
    Diagnostics := Diagnostics ∪ G
    для каждой пары (k', p') ∈ U:
        q := MergeReachable(R(k'), p')
        если q ≠ R(k'): R(k') := q, enqueue(W, k')
    C := MergeStore( $\widehat{\text{Store}}$ , Δ)
    для каждой ячейки c ∈ C:
        для каждого k' ∈ Dependencies(c):
            enqueue(W, k')
```



Из чего складывается интерпретатор

1. **Семантика** задаёт состояния и одношаговые переходы языка.
2. **Абстрактные домены** представляют множества значений и поддерживают корректные операции.
3. **Адреса и время** ограничивают память и задают контекстную чувствительность.
4. **Итерационная стратегия** вычисляет корректную префиксную точку.



Локально корректные операции всех уровней совместно образуют корректный глобальный оператор достижимости.



Список литературы

- [1]. P. Cousot.
Principles of Abstract Interpretation.
MIT Press, 2021.
- [2]. X. Rival, K. Yi.
Introduction to Static Analysis: An Abstract Interpretation Perspective.
MIT Press, 2020.
- [3]. D. Van Horn, M. Might.
Abstracting Abstract Machines.
Proceedings of ICFP, 2010.
- [4]. A. Møller, M. I. Schwartzbach.
Static Program Analysis.
Aarhus University, 2025.
- [5]. F. Nielson, H. R. Nielson, C. Hankin.
Principles of Program Analysis.
Springer, 2005.

